Mid-Year Break | 27/06 – 21/07

Monday 7th July:

Objective:

- There seemed to be a few issues with the current implementation of our testing 3x3 systolic array. Each processing element had an input of an accumulator. And the accumulator was passed from PE to PE across the array, and each PE added its produce A*B, forwarding the updated accumulator through the stream of elements.
- The issue here is that each PE is responsible for accumulating only its own output, passing the accumulator as well will cause issues, further corrupting the resultant matrix. This was indicated and proved in modelsim.
- Since the fundamental law for systolic array is that each PE must compute one output, the accumulator input for the PE code was removed and each PE will simply maintain its own accumulator (the result register) and only forwards data and weight to its neighbors.
- Previously we also used a case statement to manually assign data and weight at fixed clock cycles (using signals like *PE00_data, PE00_weight etc.).* And then manually injected the next matrix elements at specific cycles. This was kind of hardcoded for 3x3, which does make sense since we currently are only testing that size. However some changes were in order if we want to try and make the code a bit more adaptable once we make it reconfigurable. Plus it did cause some bugs such as injecting the lower rows and columns too early, causing misalignment.
- The fix for this was simply adding shift register logic:
    - Data_shift(0) receives row 0 immediately, data_shift(1) receives row 1 after 1 cycle, data_shift(2) receives row 2 after 2 cycles.
    - Same goes for weight shift(0)
- This aligns input feeding with the pipeline flow of the systolic array.
- Result valid was causing me some issues where it prevented data propagating, so I removed it, not sure if that was the issue but issues did resolve
- The contains_x function was a good idea to check for undefined bits however that needs to be implemented on an external level as the systolic array's job is not to check but to simply compute no matter what. All the external controllers surrounding it will determine and alter what goes in.
- Now the biggest change would be the new process inside the systolic array. The purpose of this new process is to control how the inputs are fed into the systolic array over time ensuring that each PE receives it's inputs at the correct clock cycle. Uses the shift register logic to delay the rows and columns, essentially mimicking the behaviour of a systolic array.

    - Upon reset, the cycle count is set to 0 and the shift registers are also set to zero, clearing the internal pipeline.
    - And then on each rising edge the cycle count is incremented by 1, keeping track of which stage of the computation we are in and then updates the shift registers accordingly to implement the feeding of rows and columns in a staggered fashion. And without this staggered feeding, inputs will arrive at the wrong time and some PEs will have already started accumulating too early or too late, resulting in wrong outcomes.
- So far this is still hard coded as it is designed for a simple 3x3 which means there are 9 explicitly instantiated PEs with 9 hardcoded connections. But it makes sense for a proof of concept to understand the principles. The next step would be to have it scaled to NxN and naturally reconfigurable.
    - To do this we could possibly use VHDL "generate" statements that can alter synthesis to instantiate different array sizes.
    - The staggered inputs is the issue here as it needs to alter the delay so that it can adapt to any size of array.
    - Essentially make a lot of signals and variables 'Dynamic'
    - Also, a control unit will be helpful as a starting point as well since that will be used in the actual NPU to generate the correct signals in the correct sequence and will probably connect to NIOS??
    - And once that stuff is onboard, it would be helpful to also make DMA controllers because that is what will feed in the data.

NxN	Clock Cycles
1	1
2	4
3	7
4	10
5	13
6	16

Based on this table, we know that the formula is: **3n-2**

Monday 14th July:

Objective: Doing deeper research into the importance of handling sparsity in systolic array inputs for our implementation.

- Activations:
    - Sparsity appears in activations due to the Rectified Linear Unit (ReLU) since it sets all negative values to zero.
- Weights:

- Sparsity in weights comes from pruning

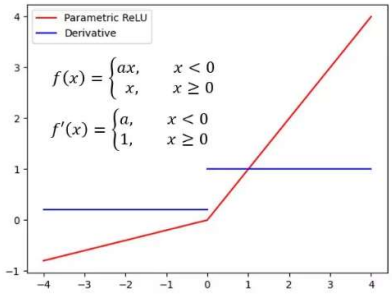
Either way, sparsity is bad because we do not want zeros even entering through the PEs as it will waste MAC resources.

Ideas to prevent sparsity:

- Implement a zero-detection system

What exactly is the Rectified Linear Unit??

The ReLU is a type of activation function used in neural networks. Defined as:



They are used in between layers of the neural network to solve the vanishing gradient issue when it comes to activation functions.

To explain simply:

- When you train a neural network, you show it an example and the correct answer. The network makes a guess, and you adjust the weights inside the neural network a little bit depending on the error so that the next guess is better.
- **But how does it know which weights to adjust?** At the end of the network, the output layer exists which is useful to see how wrong you were and push the numbers to make it closer but in a deep network, the earlier layers also need to adjust their numbers. But to figure out what they should do, the network sends a signal backward through the layers (backpropagation).
- The signal you send backward gets weaker each time it goes back through a layer. If it is too “weak”, by the time it reaches the first layers, it’s almost zero. And when the signal is close to zero, those layers don’t know how to improve, they “stop learning”. That’s the **Vanishing Gradient Problem.** Because each layer multiplies the signal by some number between 0 and 1 (depending on the math of the layer). If you multiply small numbers many times in a row, you get an even smaller number.

The steps from training to inference is something like this:
1. Initialise the model:

Defining all the characteristics and initializing the weights

2. Prepare the data

- Collect training dataset

3. Forward Pass:

- Input flows through the layers and each layer computes weighted sums.
- Activation function applied -> produces activations (some zeros = sparsity)

4. Compute Loss:

- Compare output prediction to ground-truth label using loss function

5. Backward pass

- Compute gradients of loss with respect to weights using backpropagation
- Gradients propagate back through layers -even where activations are zero.

6. Update Weights:

- Adjust the weights based on some optimization technique (not covered in this research)

REPEAT.

So the ReLU matters because it makes many activations zero. That's why activation sparsity exists. It should be fairly hardware resource friendly as it's just a mux and comparator.

Why do we care more about sparsity at inference time?

During inference, the model is deployed in a resource constrained setting, so exploiting sparsity becomes quite useful to save energy.

But Why is ReLU applied in inference as well?

Even though you're not doing backpropagation anymore, the layers still include ReLU activations. At inference time, you still want the nonlinear behavior of ReLU to produce meaningful outputs. Essentially, the model's behavior should match what was trained.

How are activations changing during inference

Due to the inputs changing, the sums change accordingly. The network is just applying the learned weights to new inputs to produce outputs. The activations always depend on the current input, they're not stored or learned, they're just the result of the computation.

The **SCNN** paper is useful as it is designed specifically for inference on CNNs, on hardware accelerators, exploiting both weight and sparsity. The way it handles sparsity is by keeping compressed representations (*compressed encoding*) of weights and activations throughout most of the pipeline. And it avoids zeros from memory or sending them to multipliers so that not only saves energy but improves the throughput.

Performance findings from this implementation:

- achieved a 2.7* speedup and 2.3* energy efficiency over dense designs at high sparsity levels.
- Since dense designs waste energy and time computing on zeros, SCNN starts to outperform dense architectures as soon as sparsity (both weights & activations) is higher than ~15%

ReLU induced sparsity dominates the activations (often > 50% zeros) during inference. So, the systolic array must be required to take advantage of the zeros produced by ReLU.

My Thoughts??

Well, if we want to have a reconfigurable NPU we could probably implement a skipping logic of the MACs when the input or weight is zero. This would be a starting point because it would only make it energy efficient. It would also be nice to reduce the time taken by redesigning the data flow so that only active nonzeroes flow into the array because we don't want to even activate any multipliers.

Could possibly consider some sort of compressed storage idea like the one in the SCNN paper. Why?

Since there are lots of weights and activations that are sparse, storing all the zeros in memory is wasteful. Memory bandwidth is wasted reading zeros; MAC units are wasted multiplying with zeros and power is wasted moving zeros around. So, you should store and send the non-zero values which can be done in various compressed formats.

So how does the NPU get streamed?

Two things are supplied for each layer:

1. Activation Inputs: the feature map(s) output by the previous layer, or the raw input image. (Essentially the data matrix that I currently have in my implementation.
2. Weights: the trained filter/kernel weights for the current layer.

The systolic array does not know or care about training. It is just performing a stream of MAC operations in the configured shape that is programmed in. The inputs will come from the on-chip memory, which is currently being designed, the host activation will come from any sort of input device. Since this is a CNN, it is most probably a sort of detection camera system.

Changes made to get closer to dynamic reconfigurability:

Now, the control unit is being written in VHDL now and so far it involves N*N inputs instead of a set size and on each clock cycle it feed staggered elements of the matrices into data shift and weight shift registers. It updates a *PE_enabled_mask* based on configured size (currently static but could become programmable) and increments *cycle_count*. It also outputs a staggered data shift and weight shift arrays for left/top feeding and enabling mask and cycle count.

For the changes in systolic array

- A grid of N×N PEs, connected horizontally (data) and vertically (weight).
    - Each PE:
    - Receives its inputs from the left & top
    - Passes its data & weight right & down
    - Accumulates locally
    - Only operates if en=1

Added a top-level systolic array entity to combine the control unit and systolic array into one entity essentially

The project requires a softcore processor NIOS. However, it is unclear what the use case would be. My guess now would be that the main controller system should be designed via software. Where the job of it would be to handle:

- Handling the in and out for DMA, like weights from external memory to on-chip memory.
    - FIFO buffers / BRAM would preload the weights and feed them column by column into the weight shift registers.
    - NIOS can program the DMA to transfer activations directly into buffers without CPU intervention and the shift registers that are currently implemented in the Datapath pipeline will take care of the rest.
- Writing activations into input buffers.
- Configuring activations into input buffers.
- Configuring the size and mode of the array (reconfigurable part)
- Starting the computation, monitoring process.
- Reading the result and triggering the next layer.

Or possibly make it so that there is a hardware control unit that will manage the pipeline, enable flags, complete handshake signals and drive the shift registers.

Week 1: Preparing for mid-year report and seminar

Monday 21th July:

Reading: Sense: Model-hardware codesign for accelerating sparse CNNs on Systolic arrays.

- This paper involves the improvement of inference performance, but it also includes model-side changes during training (which is not in the scope of our research, but will still include in notes)
    - The main problem that 'sense' tackles is that since systolic arrays are highly efficient for dense matrix multiplications, they also perform poorly when dealing with sparse CNNs. This is due:
        - Imbalances as some PEs might be idle as systolic array does not have a skip mechanism by default.
        - Compressed sparse representations do not map well to fixed hardware arrays.
        - Bandwidth and memory usage is still high if sparsity is not exploited properly

1. Model-Side: Load-Balanced Kernel Pruning
    - In typical CNN pruning, kernels can end up with uneven sparsity: one kernel might keep 70% weights while another has only 10%
    - On a systolic array, each PE computes one kernel/channel – uneven work = wasted resources
    - The solution provided by the paper:
        - Constrain training & pruning so that each kernel has the same sparsity level.
        - Algorithm:
            - Rank weights in each kernel by magnitude or importance.
            - Globally prune weights in each kernel evenly up to target sparsity.
            - Retrain to recover accuracy.
        - For our implementation we would have reconfigurable masking, and the workload would already be load-balanced, all PEs stay busy and none idle waiting for "dense" kernels.
2. Model-Side: Activation Channel Clustering
    - After ReLU, activations are sparse, but sparsity varies greatly across channels.
    - If channels are assigned randomly to rows in your array, some rows may see all-zeros.
    - The Solution from the paper is:
        - Dynamically measuring non-zero counts per activation channel
        - Group channels with similar sparsity into the same rows.
        - Result: Balanced rows, higher PE utilization.
3. Hardware-Side: Bitmap Compression
    - Memory bandwidth is a bottleneck
    - Sparse IFMs and weights are stored as:
        - Bitmap: 1-bit per element to mark zero/nonzero

        - Data: the nonzero values only
    - On-chip decoder reconstructs full streams before feeding PEs
    - Reduces DRAM traffic and buffer size requirements
    - Key takeaway: Make a decompressor on the FPGA that expands compressed IFM/weight streams into the shift registers possibly??
4. Hardware-Side: Adaptive Dataflow Modes
    - Not all layers benefit from the same data reuse strategy.
    - They define:
        - RIF (Reuse IFM first) – Keeps IFM stationery and streams weights.
        - RWF (reuse weight first) – Keeps weights stationery and streams IFM
    - Controller chooses mode at runtime depending on layer shape
5. Experimental Results (just for reference)
    - Benchmarked on ResNet, VGG, MobileNet CNN models
    - PE utilization improved from ~50% to ~85%
    - Energy efficiency improved up to 2.5x.
    - Accuracy loss < 1% with careful pruning.
    - DRAM bandwidth dropped due to compression

Week 2: Clear the current doubts and start sparsity handling

Monday 30th July:

Current Doubts:

1. How does the compression method work for sparsity in the SENSE paper

Understanding the compression method for sparsity:

- Instead of a whole 3x3 matrix for example, it is stored in a list of non-zero values alongside position info (co-ordinates).

- Essentially keeping the mathematical meaning of the matrix, the same, because during computation, the controller will know that it has to multiply e.g. value X at position (i,j) with whatever is at position (j,k) in the second matrix.
- The sparse elements of the matrix are skipped (not loaded into the PE). The logic simply ignores them, and only valid computation pairs are scheduled
- Skipping logic in the PEs disable MAC operations when zeros are encountered.

Where does sparsity occur in the pipeline?

Weight:

- Static and known at inference time
- Caused by model pruning
- These sparsity patterns do not change; they are fixed once the model is trained.

Activation:

- Layer-dependent
- Caused by ReLU, which zeros out negative values at runtime.
- After each layer, the resulting output (you C) may contain a lot of zeros
- Must be handled at each layer

What does it mean that input sparsity is dynamic at runtime?

Activations of the input matrix can change every time the model is run depending on the input data. (e.g. I feed an image of a cat, and the input feature map after ReLU is different compared to a different image (like a dog) where the ReLU might zero out at different positions.

In both cases, where the zeros are is different, so hardware must check on the fly.

How necessary is it to apply pruning after training (pre-processing)

It only needs to be done once after training. It does not need to be done for proper inference, but it can make the NPU much more efficient.

What exactly comes from the CNN model onto the systolic array NPU for inference?

During inference runtime, mode is already trained and stored on the system. There is an "input" fed in (e.g. an image like in our case since we are doing CNN). NIOS would send a small matrix window of the activation feature map (input) and a matching kernel (weight) for the current layer. Systolic array performs the convolution. The result goes through ReLU and that becomes the input for the next layer. After the systolic array, after the hardware computes the output matrix goes to the processor via shared memory, the processor do some additional applications to that input if needed and feed it to the next layer which would be another hardware accelerator block.

Why is ReLU part of training and inference

During training, ReLU is used after each layer to introduce non-linearity, ensuring only positive activations flow to the next layer. It affects the backpropagation (gradient flow), where negative gradients are stopped and causes sparsity in gradients as well.

For instance, ReLU is applied in the same position, but there is no gradient, just forward pass, and sets any negative activations to 0 which would cause sparsity in later years.

Tuesday 31th July:

Week 3&4: BRAM Implementation Attempt

Initially, matrices were being fed directly into the systolic array. This works completely fine; however, it is not scalable nor is it suitable for real operation for a reconfigured NPU. We needed the systolic array to fetch from the on-chip memory. This led to the integrating Block RAM (BRAM), instantiated as DataBuffer and WeightBuffer, each preloaded from .mif files containing the activation and weight matrices. Given success in producing such buffers would allow the handling of larger matrices without any manual hardcoding and actual NPU realism, ultimately matching how the FPGA would operate in a deployed NPU as it fetches activations and weights from memory.

Two BRAM blocks were instantiated, DataBuffer for activations (input matrix), and WeightBuffer for weights. Both were addressed in row-major order, with row and col counters generating the linear index idx = row * N + col

Each read cycle would fetch one data and one weight element which were then stored into the local 2D arrays (*matrix_data_sig*, *matrix_weight_sig*) for the control unit. Once all elements were read, a loadingCompleteFlag was raised, and *start_sig* was asserted to begin systolic array computation.

The biggest challenges faced:

1. The first one was the BRAM latency. This is when BRAM outputs are not valid in the same cycle that the address is applied, causing the first matrix elements to appear as zero, and later the final matrix element was lost (which is still unexplainable).

Many attempts to fix this problem was made:

- such as using state machines to explicitly separate address and capture cycles
- even introducing handshake signals to ensure alignment.

of these attempts, they often introduced other bugs such as repeated values, zeros at the start or the end, or even missing rows.

2. Another issue was synchronizing with the control unit. The control unit required a clean start signal only once the matrices were fully loaded. However, earlier versions caused systolic processing to begin with incomplete data. This wasn't all that big of a problem as the solution was only to introduce a flag and a delayed start signal to ensure that the systolic array only ran after the final element was captured.

This part of the implementation was unsuccessful by me and proved to be quite difficult for when it cam to debugging as the reports provided in the modelsim transcripts provided final results and not the intermediate data flow which is why debug ports were added to view that. The current status of the BRAM implementation is that the loader is successfully building the local matrices from BRAM with minor misalignment issues at the first and last elements but due to that the final results are not matching due to mathematical errors resulting from those misalignments. Hence why I am delegating this task to my project partner effective immediately to focus our soft-core processor and main algorithm implementation.

Week 6 & Mid-Term Holidays:

Read papers for sparsity handling

Week 7 (15/09/25):

- Had Meeting with Morteza after long time

Week 8:

24/09

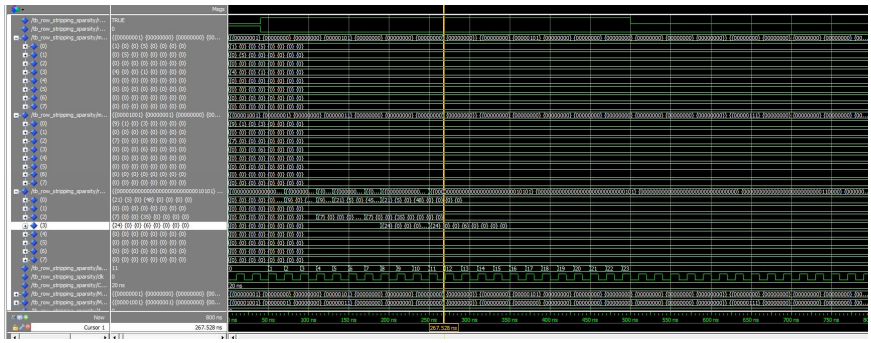
- - row sparsity handler works
- - CU is functional as it is. It now has two new signals, active row and col which generate the staggered streams and the PE enable mask for any matrix size within our defined grid size (8x8)

- - the python script is what automatically takes a large sparse matrix, strips the matrix of its empty rows and generates the exact VHDL constants that i can just paste into the TB which connect it through to the top level design
- - i believe one of the matrices that i tested had 8 cycles instead of 27. for this example:
- Data [[9, 1, 0, 3],
- [0, 0, 0, 0],
- [7, 0, 0, 0],
- [0, 0, 0, 6]]
-

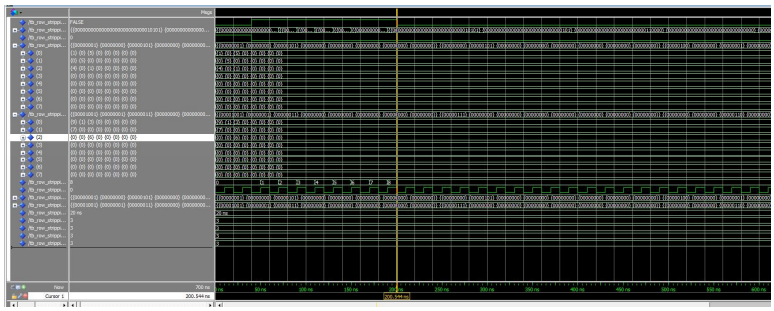
- Weight [[1, 0, 0, 5],
- [0, 5, 0, 0],
- [0, 0, 0, 0],
- [4, 0, 0, 1]]
-
- - in terms of energy savings, no clue how that would be tested but you can testby checking how many PEs are activated (MAC operations). The system exists with the PE mask that disables the rest of the PEs if it is a 4x4 within the large 8x8 for example. which is what reduces the energy consumption, however it would still need to traverse through the entire MAC engine in an NPU. That is why a "vanilla" calculation would be 27 cycles because in an NPU even if the outer parts of the actual matrix within let's say a 256 by 256 grid is deactivated (the PEs), it would still need to traverse through that unless and until there is a software based algorithm to make that efficient, which is what we are attempting

- Row/col removal sparsity handling works, systolic array architecture is reconfigurable for that (N*N) and (N*M),
- However since the CU only takes in row and col of one array right now rather than info from both arrays, it does not do the most efficient throughput for the rectangles, as you can see in the last set of images I sent for the big 8x8, the most efficient throughput for that would be 8 cycles but as i said i am missing a parameter in the CU, which is doable, i can add it tomorrow,
- Started off making a UART core by hand with online research examples, realised that was not smart because it already exists in the IP catalog, did that, fixed some bugs, synthesised, added pin assignments to test some data transfer to turn on an LED. Realised that i can't plug my computer into it because the UART to USB port on the FPGA board is hooked onto the ARM chip and i would need to compile a C program alongside an OS booted on it just to even use that. However, I can just use a USB to TTL adapter which will just connect to the GPIO pins via female jumper cables and then connect to duh computer and we continue as planned.,
- I will ask Morteza if he's okay with me continue with the UART idea or he wants more on the algorithm since very little time, otherwise i shall find out where I can take an adapter from in uni.

```
# ** Note: --- PERFORMANCE REPORT ---
#    Time: 600 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity
# ** Note: Active Dimensions (Data): 8x8
#    Time: 600 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity
# ** Note: Measured Latency: 23 cycles.
#    Time: 600 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity
# ** Note: -------------------------
#    Time: 600 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity
```

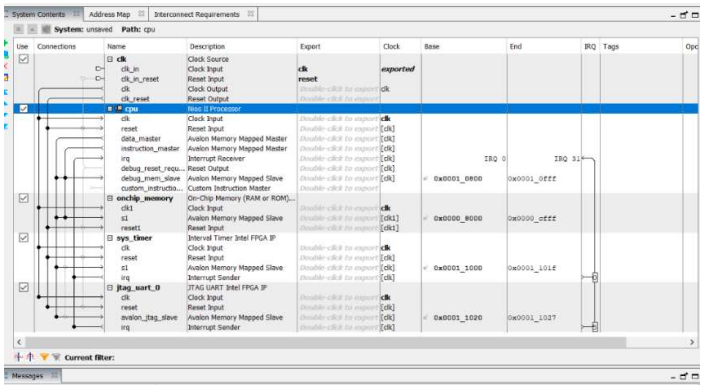


```python
# Define original sparse matrices in Python
N_hardware = 8
inputMatrix_data = np.array([[9, 1, 0, 3],
                             [0, 0, 0, 0],
                             [7, 0, 0, 0],
                             [0, 0, 0, 6]])

inputMatrix_weight = np.array([[1, 0, 0, 5],
                               [0, 5, 0, 0],
                               [0, 0, 0, 0],
                               [4, 0, 0, 1]])
```



```
VSIM 23> run 700 ns
# ** Note: --- PERFORMANCE REPORT ---
#    Time: 300 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity
# ** Note: Active Dimensions (Data): 3x3
#    Time: 300 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity
# ** Note: Measured Latency: 8 cycles.
#    Time: 300 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity
# ** Note: --------------------------
#    Time: 300 ns  Iteration: 0  Instance: /tb_row_stripping_sparsity

VSIM 24>
```

25/09



- Trying to make a wrapper using platform designer with the top_level_systolic_array as a custom component inside the designer which would essentially hold the actual NPU hardware.
- The purpose of this attempt is to be able to eventually have a soft-core processor running that would manage the memories.

When synthesising onto the board, the code has issues where we can't compile it

because of pin overflow.

• DE1-SOC has 256KB memory.

o Input: 64 element matrix * 2 matrix * 1 byte (int8) = 128 Bytes

o Output: 64 element matrix * 4 bytes (int32) = 256 Bytes

o Totalling = 385 Bytes

AlexNet:

• Eight Layers: 5 convolutional, some max-pooling layers, three fully connected layers

• The avgpool flattens the activation layer

Issue with the current Stripping Algorithm:

• The issue comes from the fact that we are doing row stripping on both. For hatever reason, when we row strip BOTH the activation and data matrices, there is mismatch happeningbetween the multiplication.

• Looking at the first PE, row of A, and column of B alone:

- We are expecting the behaviour of 90x13 + 90x4 + 90 x 27 = 3960. However, because we have stripped a row in data, we get 90x13 + 90x4 + **92**x27 = 4014
- This indicates what when we row strip from the data matrix, the activation matrix's column is somehow one column behind

Algorithms now consider both matrices at the same time when getting rid of a row/column. What does this is if it gets rid of a row in Matrix A, it checks if we get rid of the column in Matrix B. This maintains the positions correctly so we can correctly apply the MAC operation in the systolic array.

The downside to this (compared to the original algorithm) is that there is less row/column stripping happening. This means that we are not reducing the clock cycles as much as we hope, but at least we know it could work for real matrices. An important note is that since the activation matrix is always going to be 1x8, the weight matrix will always be 8x8 UNLESS we get rid of a column

This is because we can't get rid of rows in the weight matrix since there is a number in all the columns of the activation matrix. So, in terms of reduction, we must hope that there is an empty column in the weight.

Week 9:

- When working with the 32-bit NIOS controller, if possible, it would be nice to have a 32-bit wide BRAM.
- Attempted to pack four 8-bit element matrix elements into a single 32-bit word.
- Currently trying to make a hardware FSM to read one 32-bit word at a time to unpack it
- Based upon researching the Cyclone V M10K blocks we noticed that the Avalon MM interconnect is optimized for 32-bit transactions, especially when dealing with 32-bit data bus of the NIOS II processor.
- It could translate to a theoretical 4x reduction in data loading time.
    - For an 8x8 matrix it is 64 bytes, an 8-bit interface requires 64 read cycles, whereas our 32-bit interface requires only 16 read cycles
- Challenges I came across:
    - Python script was generating elements incorrectly due to bad code generation
    - The bitwise shifting logic used for packing was unreliable, due to some type-handling issues
    - To address the address issue, the python struct library was used.
    - Initial modelsim simulations were a complete failure. The result_matrix would fill with unknown values before the input values were even loaded, and the simulation would terminate prematurely.
    - The FSM was asserting the npu_start signal at the same time as it was trying load data from the BRAMs. Since the RAM has 2-cycle read latency the FSM has only one waiting one cycle, meaning it was consistently reading data inconsistently (from previous memory access). The sequence had states like SET_ADDR, WAIT, LATCH

- This ended up generating the inputs correctly but then the systolic array staggering ended up being implemented incorrectly so the output was wrong, so we decided to put down this idea for now as time was running out.

NIOS Side of things:

- Had so many issues:
    - Went from trying hardware UART to trying JTAG uart on NIOS to establish the connection with python. All of which did not work out.

Week 10

We put down the pen for NIOS and other memory stuff for now.

The goal right now is to attempt to connect the MIF to simulation

Instantiate the buffers in the TB

Read the buffers separately. Store these values into an array. Create an FSM with START, READ, END states. Test with a simple 8x8 array. Test with a sparse matrix. Hardcode the active_rows and active_cols values into the TB itself. Can later add this to the MIF if required / have the time.

control unit now has a ready signal, asserted when we are done reading the MIFfiles. This is because it will start counting as soon as rising edge starts, meaning that we are not accessing the matrix in time. 3N-2 is the theoretical clock cycle formula, however, since most of the testing we have done are with 1D arrays (post sparsity processing), this formula has become 2n. NOTE: this is only for 1D arrays

we found that the stripping algorithm does not remove any sparse row or column. when the algorithm is deciding whether or not to strip a row or column, it must check the if the inner dimensions are. for example, if the data matrix can be compacted down to 5x3 but the weight matrix is limited to 8x8, the algorithm does not strip the columns in the data matrix. This means that the size will still be 5x8 for oblige with matrix multiplication rules This means that in modelsim, we will still process those zero columns, but

Since we are accumulating zeros to the PE, the results matrix will stop updating much earlier on. Although it might not update its value on modelsim, it is important to note

that these processes are still happening. This may not be the most efficient algorithm but the latency is still less than 3N-2+1. For a completely sparse data matrix, the latency is still 15 clock cycles. This feels wasteful but it is better than 23.

- Doing a bunch of testing for the poster and report:

Create an NPU wrapper to be synthesised to Quartus
Observations:
• MAC Count: counting the active PEs in the output (i.e. output dimension – PE with zero output)
o Noticed that none of the tested output contained zero in the output dimension
• NPU Wrapper:
o Instantiated top_level_systolic_array and the data buffers
o Because we have limited pins in DE1-SoC, we cannot map the systolic_array 8x8 output to the GPIO pins
o We modified this by sending the 32-bit value through an output bus
• Compilation Report:
o No DSPs are used as design is simple and small
o Processing Element no longer does zero-gating.
▪ This sped up compilation from 4:12 to 3:30. We can get away with this because accumulating zero does not change the result AND using comparators for the if condition is costly for hardware.
▪ Furthermore, the active matrix is sparse, but the weight matrix is dense. This means that the output matrix (as mentioned above) will never be zero. Although accumulating zero is redundant, going through the if condition and doing the MAC operation is done in the same clock cycle
▪ Since this does not change the timing analysis or hardware utilisation (in terms of the slack value), then this decision makes more sense
o Uses 1 RAM block: both data and weight buffers use 128 words x 8 bits. The size of 1 M10K block is 1024 words x 8 bits
o No matter file we test, Quartus will synthesise it worst case. Since we are always instantiating the systolic array to be 8x8, even though we are PE-gating, it will synthesise it as 8x8 regardless

-- Sparsity Analysis Summary ---
Total images processed: 156
Average Original Latency (dense 8x8): 22.00 cycles
Average Compacted Latency (after stripping): 9.07 cycles
Latency Reduction: 58.75%
--- ALEXNET MODEL PROFILING ---
Total MACs for AlexNet: 0.71 GMACS
to conclude:

a latency reduction of 58.75% would mean that the NPU would only use 41.25% of the original cycles
Acceleration factor: 1/0.4125 = 2.42

Week 10 (weekend)

Kristelle made the hardware UART drivers work:

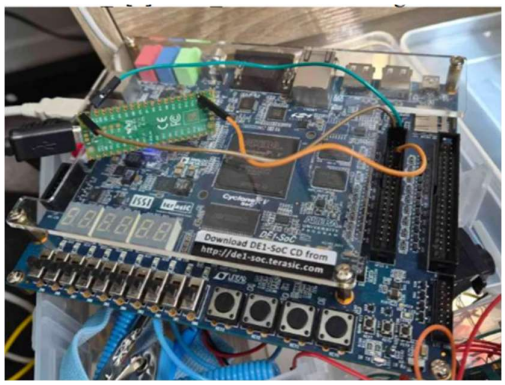
HPS / USB-UART:

• Has a mini USB-B port to USB-A port

• Requires the ARM processor

• Hard to configure and time consuming

• Configured QSF file to connect to all the peripherals and GPIO



xlviii

After many hours of debugging, we finding issues in NPU_wrapper, the most recent configuration of UART is:

• UART can send 8 bits / 1 byte

• GPIO_0[0] / PIN_AC18 has been assigned to UART RX and GPIO_0[1] / PIN_Y17 has been assigned to UART TX

• Pressing KEY[1] will make the FPGA send data through UART

• The data sent through is the clock cycles it took for the Systolic Array to do its computations

• The FPGA will send this and the Pico will calculate the time taken by multiplying the clock cycles by 1/50MHz.

```
C:\Users\iamKr\Documents\part-4-project\Final\demo>python demo.py
 Average Duration for tile 0: 5678.40 ns over 1000 runs, where M = 0, N = 8, K = 8
 Average Duration for tile 1: 5395.10 ns over 1000 runs, where M = 1, N = 8, K = 8
 Average Duration for tile 2: 5577.90 ns over 1000 runs, where M = 2, N = 8, K = 8
 Average Duration for tile 3: 5679.50 ns over 1000 runs, where M = 3, N = 8, K = 8
 Average Duration for tile 4: 5877.50 ns over 1000 runs, where M = 4, N = 8, K = 8
 Average Duration for tile 5: 6166.60 ns over 1000 runs, where M = 5, N = 8, K = 8
 Average Duration for tile 6: 6129.80 ns over 1000 runs, where M = 6, N = 8, K = 8
 Average Duration for tile 7: 6152.30 ns over 1000 runs, where M = 7, N = 8, K = 8
 Average Duration for tile 8: 6295.50 ns over 1000 runs, where M = 8, N = 8, K = 8
```

```
charan@sandevistan:~/Downloads/demo$ python3 demo.py
Average Duration for tile 0: 1714.82 ns over 1000 runs, where M = 0, N = 8, K = 8
Average Duration for tile 1: 1720.29 ns over 1000 runs, where M = 1, N = 8, K = 8
Average Duration for tile 2: 1752.57 ns over 1000 runs, where M = 2, N = 8, K = 8
Average Duration for tile 3: 1818.95 ns over 1000 runs, where M = 3, N = 8, K = 8
Average Duration for tile 4: 1834.57 ns over 1000 runs, where M = 4, N = 8, K = 8
Average Duration for tile 5: 1904.39 ns over 1000 runs, where M = 5, N = 8, K = 8
Average Duration for tile 6: 1922.63 ns over 1000 runs, where M = 6, N = 8, K = 8
Average Duration for tile 7: 1936.08 ns over 1000 runs, where M = 7, N = 8, K = 8
Average Duration for tile 8: 1967.15 ns over 1000 runs, where M = 8, N = 8, K = 8
charan@sandevistan:~/Downloads/demo$
```